

Resumen Parcial 2 — Ingeniería de Software

| **Abarca:** Clase 7 (desde Métricas de Especificidad y Completitud) → Clase 13

ÍNDICE

1. Métricas de Especificidad y Completitud (Clase 7)
 2. Métricas para el Modelo de Diseño (Clase 7)
 3. Métricas para el Modelo de Implementación y Pruebas (Clase 7)
 4. Planificación de Proyectos de Software (Clase 8)
 5. Estimación: Técnicas de Descomposición (Clase 8)
 6. COCOMO II (Clase 9)
 7. Estimación con Casos de Uso — Use Case Points (Clase 9)
 8. Calendarización del Proyecto (Clase 10)
 9. Administración del Riesgo (Clase 11)
 10. Planificación de la Gestión de la Calidad (Clase 12)
 11. Planificación de la Gestión del Cambio (Clase 12)
 12. Modelos Formales y Estándares (Clase 13)
-

1. Métricas de Especificidad y Completitud

(Clase 7 — segunda parte)

Estas métricas evalúan la **calidad de los requerimientos** antes del diseño.

1.1 Especificidad (falta de ambigüedad) — Q1

Mide qué tan claro e inequívoco es cada requerimiento para todos los revisores.

$$Q_1 = \frac{nui}{nr}$$

- **nui** = número de requerimientos para los cuales **todos** los revisores tienen interpretaciones idénticas.
- **nr** = número total de requerimientos funcionales y no funcionales.

| **Interpretación:** Mientras más cercano a 1 esté Q1, menor es la ambigüedad. Q1 = 1 significa que todos los requerimientos son claros para todos.

1.2 Completitud de los requerimientos — Q2

Mide qué proporción de requerimientos ha sido validada correctamente.

$$Q_2 = \frac{nc}{nc + nnv}$$

- **nc** = número de requerimientos validados como correctos.
- **nnv** = número de requerimientos que **no** se han validado.

Interpretación: Una especificación es completa si todo lo que el software debe hacer está incluido, contemplando todas las entradas posibles, todas las salidas posibles y todos los escenarios posibles. Q2 = 1 es el valor ideal.

1.3 Métricas para Requerimientos No Funcionales

Atributo	Métricas
Rapidez / Tiempo de respuesta	Transacciones por segundo; Tiempo de respuesta al usuario
Facilidad de uso	Tiempo de formación; Cantidad de cuadros de ayuda consultados
Fiabilidad	MTBF = Tiempo total operación / Nº de fallos; Tasa de fallos (fallos/hora); Disponibilidad = (Horas esperadas – Horas de mantenimiento) / Horas esperadas
Tamaño	Capacidad de almacenamiento máxima (ej. 10 TB)
Seguridad	Tiempo Medio de Detección = Σ Tiempo de detección de cada incidente / Nº de incidentes

2. Métricas para el Modelo de Diseño

(Clase 7 — segunda parte)

2.1 Métricas del Diseño Arquitectónico

Para arquitecturas jerárquicas se definen tres medidas de complejidad:

Complejidad estructural:

$$S(i) = f_{out}(i)^2$$

donde $f_{out}(i)$ es el **fan-out** del módulo i (cantidad de módulos que invoca directamente).

Complejidad de datos:

$$D(i) = \frac{v(i)}{f_{out}(i) + 1}$$

donde $v(i)$ es el número de variables de entrada y salida que pasan hacia y desde el módulo i .

Complejidad del sistema:

$$C(i) = S(i) + D(i)$$

Ejemplo: Si el módulo `clientes` tiene fan-out = 2 y $v(i) = 21$:

- $S(\text{clientes}) = 2^2 = 4$
- $D(\text{clientes}) = 21 / (2+1) = 7$
- $C(\text{clientes}) = 4 + 7 = 11$

2.2 Métricas Orientadas a Objetos — Suite CK (Chidamber y Kemerer)

Seis métricas de diseño basadas en clase para sistemas OO:

Nº	Métrica	Descripción
1	MPC — Métodos Ponderados por Clase	$MPC = \sum Ci$ (suma de complejidades de cada método). Mayor MPC → mayor complejidad.
2	PAH — Profundidad del Árbol de Herencia	Mayor PAH → más métodos heredados → mayor complejidad y mayor potencial de reutilización.
3	NDH — Número de Hijos	Mayor NDH → mayor reuso, pero también mayor esfuerzo de pruebas.
4	ACO — Acoplamiento entre Clases de Objetos	Valores altos complican modificaciones y pruebas. Debe mantenerse bajo.
5	RPC — Respuesta Para una Clase	Conjunto de métodos que pueden ejecutarse ante un mensaje. Mayor RPC → mayor complejidad de diseño y pruebas.
6	FCOM — Falta de Cohesión en Métodos	Nº de métodos que acceden a uno o más atributos comunes. Si 4 de 6 métodos acceden a atributos comunes, FCOM = 4. Deseable mantenerla baja.

2.3 Suite de Métricas MOOD

- **FHM — Factor de Herencia de Método:** grado en que la arquitectura de clases utiliza la herencia para métodos y atributos.
- **FA — Factor de Acoplamiento:** a mayor FA, mayor complejidad del sistema OO.

2.4 Métricas de Lorenz y Kidd

Cuatro categorías:

- **Tamaño de una clase:** conteos de atributos y operaciones (individuales y promedio del sistema).
- **Herencia:** cómo se reutilizan las operaciones en la jerarquía de clases.
- **Métricas internas:** cohesión y conflictos orientados a código.
- **Métricas externas:** acoplamiento y reuso.

2.5 Métricas de Diseño en el Nivel de Componente (orientadas a operación)

- **TOprom — Tamaño promedio de operación:** número de líneas de código o mensajes enviados por método.
 - **CO — Complejidad de la operación:** debe mantenerse baja; cada método debe tener una sola responsabilidad.
 - **NPprom — Número promedio de parámetros por operación:** a mayor NPprom, más compleja la colaboración entre objetos. Mantener bajo.
-

2.6 Métricas de Usabilidad

- **NPS — Net Promoter Score:** escala 0 a 10.
 - 0-6 → Detractores
 - 7-8 → Pasivos
 - 9-10 → Promotores

10 preguntas de usabilidad que deben responderse con "Sí" (si el sistema es usable sin ayuda continua, si las reglas de interacción son eficientes, si el sistema informa su estado al usuario, etc.).

3. Métricas para Implementación y Pruebas

(Clase 7 — segunda parte)

3.1 Métricas de Halstead (para código fuente)

Medidas primitivas:

Símbolo	Descripción
n1	Número de operadores distintos en el programa
n2	Número de operandos distintos en el programa
N1	Número total de ocurrencias de operadores
N2	Número total de ocurrencias de operandos

Fórmulas derivadas:

$$\text{Longitud del programa: } N = N_1 + N_2$$

$$\text{Volumen: } V = N \times \log_2(n_1 + n_2)$$

$$\text{Esfuerzo: } E = V \times \frac{n_1}{2} \times \frac{N_2}{n_2}$$

$$\text{Tiempo de entendimiento/implementación: } T = \frac{E}{18} \text{ (en segundos)}$$

Ejemplo resuelto: Para el algoritmo de ordenamiento dado en clase:

- $n_1 = 10, N_1 = 23, n_2 = 5, N_2 = 22$
 - $N = 23 + 22 = 45$
 - $V = 45 \times \log_2(15) = 45 \times 3,907 \approx 175,8$
 - $E = 175,8 \times (10/2) \times (22/5) = 175,8 \times 5 \times 4,4 = \mathbf{3.867,6}$
 - $T = 3867,6 / 18 \approx \mathbf{214,87 \text{ segundos}}$
-

3.2 Métricas para Pruebas

Exactitud: grado en el que el software realiza la función requerida (ej. cantidad de defectos del sw).

ERD — Eficiencia en la Remoción del Defecto:

$$ERD = \frac{E}{E + D}$$

- **E** = errores encontrados **antes** de entregar al usuario final.
- **D** = defectos encontrados **después** de la entrega.
- Valor ideal: $ERD = 1$ (no hay defectos post-entrega).

Tasa de éxito de pruebas:

$$\text{Tasa de éxito} = \frac{\text{cantidad de pruebas exitosas}}{\text{total de pruebas ejecutadas}} \times 100$$

3.3 Métricas de Mantenimiento

MTTR — Tiempo Medio de Resolución:

$$MTTR = \frac{T_d}{N_r}$$

- **Td** = Tiempo total desde descubrimiento hasta resolución de todos los errores.
- **Nr** = Número total de reparaciones.

Si un error no puede repararse, se registra el tiempo que el sistema estuvo inactivo.

Integridad del sistema (resistencia a ataques):

$$\text{Integridad} = 1 - (\text{amenaza} \times (1 - \text{seguridad}))$$

Ejemplo: Probabilidad de amenaza = 0,20; probabilidad de repeler = 0,40 (seguridad):
 $\text{Integridad} = 1 - (0,20 \times (1 - 0,40)) = 1 - 0,12 = \mathbf{0,88 = 88\%}$

IMS — Índice de Madurez de Software:

$$IMS = \frac{M_t - (F_c + F_a + F_d)}{M_t}$$

- **Mt** = número de módulos en la versión actual.
- **Fc** = módulos con cambios en la versión actual.
- **Fa** = módulos agregados.
- **Fd** = módulos de la versión anterior eliminados.

Conforme $IMS \rightarrow 1$, el producto se estabiliza.

4. Planificación de Proyectos de Software

(Clase 8 y 10)

La planificación de proyectos abarca **cinco actividades**:

1. **Estimación**
2. **Calendarización** (fechas de entrega)
3. **Análisis de riesgos**
4. **Planificación de la gestión de la calidad**

5. Planificación de la gestión del cambio

¿Qué es la estimación?

Intento de determinar cuánto dinero, recursos y tiempo tomará construir un sistema.

Pasos generales:

1. Describir el ámbito del problema.
2. Descomponer en problemas más pequeños (requerimientos).
3. Estimar cada uno usando datos históricos y experiencia.
4. Considerar complejidad y riesgo antes de la estimación final.

Factores que afectan la estimación:

- Complejidad del proyecto (experiencia del equipo).
- Tamaño del proyecto (cuánto hay que construir).
- Grado de incertidumbre estructural (entendimiento de los requisitos).

Factibilidad — cuatro dimensiones:

- Tecnología (¿es técnicamente posible?)
 - Finanzas (¿puede pagarse?)
 - Tiempo
 - Recursos
-

5. Técnicas de Descomposición

(Clase 8)

5.1 Dimensionamiento de componente estándar

Se reutilizan datos históricos. Ejemplo: si cada módulo CRUD toma 3 horas y hay 10 módulos → 30 horas estimadas.

5.2 Estimación con Casos de Uso (Use Case Points — UCP)

Ver sección 7.

5.3 Modelos Empíricos

Ver COCOMO II (sección 6).

5.4 Estimación Ágil

En Scrum, la estimación ocurre durante el **Sprint Planning**.

Métodos:

Método	Descripción
Delphi	El grupo estima anónimamente un ítem del Backlog y se repite hasta consenso.
Story Points	Se asignan puntos a cada historia de usuario según su dificultad.
Planning Poker	Como Delphi, usando cartas con la secuencia Fibonacci: 0, 1, 2, 3, 5, 8, 13, 21.
Técnica de camisetas	Se clasifica la complejidad en XS, S, M, L, XL.

Pasos de la estimación ágil:

1. Considerar cada historia de usuario por separado.
2. Descomponer la historia en tareas de ingeniería de software.
3. Estimar el esfuerzo de cada tarea individualmente.
4. Sumar estimaciones por historia de usuario.
5. Sumar todas las historias del incremento.

Matriz de Eisenhower (priorización):

	Urgente	No urgente
Importante	Hacer (error crítico en producción)	Programar (nueva funcionalidad futura)
No importante	Delegar (corrección menor urgente)	Eliminar (requisitos triviales)

6. COCOMO II

(Clase 9)

COnstructive **CO**st **MO**del — desarrollado por Barry Boehm (1990s).

Permite realizar estimaciones en función del **tamaño del software** y un conjunto de **factores de costo y de escala**.

Opciones de dimensionamiento:

1. Puntos de función no ajustados.
2. Líneas de código fuente.

Factores de Escala (5 factores):

Factor	Descripción
Precedentes	Experiencia previa en lo que hay que construir
Flexibilidad de desarrollo	Muy bajo = proceso de desarrollo muy rígido; muy alto = cliente solo establece metas generales
Resolución de la arquitectura/riesgo	Muy bajo = poco análisis de riesgo
Cohesión de equipo	Muy bajo = interacciones muy difíciles
Madurez del proceso	De la organización

Factores de Costo (4 grupos):

- **Del producto:** características requeridas del software.
- **Del personal:** experiencia y capacidades del equipo.
- **De la plataforma:** restricciones de hardware/software.
- **Del proyecto:** características particulares del desarrollo.

7. Use Case Points (UCP)

(Clase 9) — Desarrollado por Gustav Karner (1993), supervisado por Ivar Jacobson.

Calcula el esfuerzo a partir de: actores y CU identificados + requerimientos técnicos + factores ambientales.

Paso 1 — Clasificar Actores (UAW)

Tipos de actores con sus pesos:

Tipo de actor	Descripción	Peso
Simple	API o sistema externo	1
Promedio	Subsistema / interfaz de protocolo	2
Complejo	Interfaz gráfica con usuario humano	3

$$UAW = \sum (\text{cantidad de un tipo de actor} \times \text{factor})$$

Paso 2 — Clasificar Casos de Uso (UUCW)

Se puede usar por **transacciones** (Tabla 1) o por **clases de análisis** (Tabla 2):

Por transacciones:

Tipo de CU	Transacciones	Peso
Simple	1-3	5
Promedio	4-7	10
Complejo	> 7	15

Por clases de análisis:

Tipo de CU	Clases	Peso
Simple	< 5	5
Promedio	5-10	10
Complejo	> 10	15

$$UUCW = \sum (\text{cantidad de un tipo de CU} \times \text{factor})$$

Paso 3 — Puntos de Casos de Uso No Ajustados (UUCP)

$$UUCP = UAW + UUCW$$

Paso 4 — Factor Técnico (TCF)

Se asigna un valor de influencia de 0 a 5 a cada factor técnico.

$$TFactor = \sum (Valor_i \times Peso_i)$$

$$TCF = 0,6 + (0,01 \times TFactor)$$

Paso 5 — Factor Ambiental o de Entorno (EF)

Se asigna un valor de 0 a 5 a cada factor de entorno. 1 = fuerte impacto negativo; 3 = impacto medio; 5 = fuerte impacto positivo.

$$EFactor = \sum (Valor_i \times Peso_i)$$

$$EF = 1,4 - (0,03 \times EFactor)$$

Paso 6 — Puntos de Casos de Uso Ajustados (AUCP)

$$AUCP = UUCP \times TCF \times EF$$

Paso 7 — Esfuerzo (E)

Contar los factores ambientales E1-E6 con puntuación < 3 y los E7-E8 con puntuación > 3.

- Si la suma es 0-2 → CF = 20 horas/persona por punto
- Si la suma es 3-4 → CF = 28 horas/persona por punto
- Si la suma ≥ 5 → reconsiderar el proyecto (riesgo alto)

$$E = AUCP \times CF$$

Importante: este esfuerzo cubre solo la **codificación** ($\approx 40\%$ del total del proyecto).
Para el esfuerzo total hay que sumar el resto de fases del ciclo de vida.

8. Calendarización del Proyecto

(Clase 10)

Principios básicos:

1. **Descomposición:** dividir en actividades y tareas manejables.
2. **Interdependencia:** determinar qué tareas van en secuencia y cuáles en paralelo.
3. **Asignación de tiempo:** fecha de inicio y fin, con unidades de trabajo (persona-días).
4. **Validación de esfuerzo:** no calendarizar más trabajo que el que las personas disponibles pueden hacer.
5. **Definir responsabilidades:** cada tarea asignada a un miembro específico.
6. **Definir resultados:** cada tarea debe tener un producto operativo como resultado.
7. **Hitos definidos:** cada tarea o grupo de tareas asociada a un hito (momento que indica progreso y se revisa en cuanto a calidad).

Hitos vs. Tareas

- **Hito:** momento significativo en el proyecto que indica progreso. No es una tarea; es un punto de control.
- **Pregunta clave:** ¿Es un momento significativo? ¿Afecta el plazo final? ¿Las partes interesadas deben revisarlo? → Si sí a alguna → es un hito.

Los hitos permiten: controlar plazos, mostrar fechas críticas, asignar recursos, mejorar la gestión del tiempo, facilitar el seguimiento de gastos.

Herramientas de calendarización:

- **PERT** (Program Evaluation and Review Technique): usa modelos estadísticos para estimar tiempos.
- **CPM** (Critical Path Method / Método de Ruta Crítica): determina la cadena de tareas que define la duración total del proyecto.
- **Gráfico de Gantt:** cronograma visual por tarea/función/persona.

Ambas técnicas permiten: determinar la ruta crítica, establecer estimaciones de tiempo más probables y calcular tiempos frontera (holgura de tiempo para una tarea).

Recursos del proyecto:

Cada recurso se especifica con:

1. Descripción del recurso.
2. Enunciado de disponibilidad.
3. Momento en que se requerirá.
4. Duración de su aplicación.

Tipos: recursos humanos (personas/meses + roles), componentes reutilizables (librerías, APIs, CSS), entorno de desarrollo (herramientas HW y SW, conectividad).

Seguimiento del calendario:

- Reuniones periódicas del estado del proyecto.
- Evaluar resultados de revisiones del proceso de IS.
- Determinar si los hitos se lograron en fecha.
- Comparar fechas de inicio real vs. planeadas.

¿Qué pasa si se agrega personal a un proyecto retrasado? → El nuevo personal debe aprender el sistema, y quienes le enseñan dejan de trabajar, retrasando aún más el proyecto. La recomendación es asignarles trabajo muy descompuesto o fragmentado.

9. Administración del Riesgo

(Clase 11)

¿Qué es el riesgo?

Un problema potencial: puede ocurrir o no. Dos características:

1. **Incertidumbre** (ningún riesgo es 100% probable).
2. **Pérdida / Impacto** (consecuencias no deseadas si ocurre).

Tipos de Riesgo:

Tipo	Amenaza	Ejemplos
Riesgo del proyecto	Cronograma o recursos	Pérdida de un desarrollador clave; subestimación del tamaño

Tipo	Amenaza	Ejemplos
Riesgo técnico	Calidad o rendimiento del software	Componente con rendimiento menor al esperado; ambigüedad en la especificación
Riesgo empresarial	Viabilidad del software	Construir algo que no se quiere (mercado); perder apoyo directivo; riesgo presupuestal

Los tipos no son mutuamente excluyentes. La pérdida de un programador experto puede ser riesgo del proyecto, del producto y del negocio simultáneamente.

Proceso de Gestión de Riesgos (4 etapas):



1. Identificación: listar riesgos potenciales (ej. deficiente captura de requisitos, cambios de requerimientos, desconformidad del usuario, retrasos en entregables, problemas financieros).

2. Análisis:

- Probabilidad:** muy bajo (<10%), bajo (10–25%), moderado (25–50%), alto (50–75%), muy alto (>75%).
- Impacto:** catastrófico, serio (crítico), tolerable (marginal), insignificante (despreciable).
- Se elabora una tabla de priorización ordenada de mayor a menor probabilidad × impacto.

Matriz de riesgos (color):

	Insignificante	Menor	Crítico	Mayor	Catastrófico
Constante					
Moderado					
Ocasional					
Posible					
Improbable					

3. Planeación (estrategias):

Estrategia	Cuándo aplicarla	Descripción
Prevención	Antes de que ocurra	Eliminar la causa del riesgo
Minimización	Antes de que ocurra	Reducir el impacto si ocurre
Contingencia	Si ya ocurrió	Plan de acción para recuperarse

4. Supervisión: valorar periódicamente (semanal o quincenal) si los riesgos ocurrieron, si las acciones preventivas se aplican correctamente y recopilar información para futuros análisis. Tres objetivos: verificar si el riesgo ocurrió, confirmar que los planes se aplican, recopilar datos para análisis futuros.

Hoja de Información del Riesgo (elementos clave):

- Id del riesgo, fecha, probabilidad, impacto, categoría.
- Descripción del riesgo.
- Refinamiento/contexto (subcondiciones).
- Estrategia de reducción/supervisión.
- Gestión/plan de contingencia.
- Estado, autor, asignado, versión.

10. Planificación de la Gestión de la Calidad

(Clase 12)

Objetivo

Lograr **software de alta calidad**: que satisfaga las necesidades del consumidor, con desempeño apropiado y confiable, y que agregue valor para todos los que lo utilizan.

¿Cómo lograr calidad?

1. Usar procesos y prácticas probados de la ingeniería de software.
2. Administrar bien el proyecto.
3. Realizar un control de calidad exhaustivo.
4. Contar con infraestructura de aseguramiento de la calidad.

Plan de Calidad — Estructura:

1. **Introducción del producto:** descripción, mercado, expectativas de calidad.

- 2. **Planes de producto:** fechas de terminación, responsabilidades, distribución y servicio.
- 3. **Descripciones del proceso:** procesos de desarrollo y de servicio.
- 4. **Metas de calidad:** atributos importantes, identificación y justificación.
- 5. **Riesgos y gestión de riesgos:** riesgos que podrían afectar la calidad y las acciones para abordarlos.

El plan de calidad debe ser lo más compacto posible. Si es muy grande, los ingenieros no lo leerán.

Plan de Aseguramiento de la Calidad del Software (SQA)

Define las actividades mínimas para alcanzar la calidad. Abarca:

- Especificación formal de requerimientos.
- Modelos de diseño.
- Código y tests generados.
- Documentación desarrollada.

Revisiones y auditorías por fase del ciclo de vida:

Fase	Revisiones y auditorías
Especificación de requerimientos	Revisión de la Especificación; Auditoría del proceso (opcional)
Diseño	Revisión del diseño preliminar; Revisión del diseño detallado; Revisión del plan de pruebas; Revisión de especificación y procedimiento de prueba
Implementación	Revisión del código; Revisión de la prueba de unidad
Integración y prueba	Revisión de las pruebas; Auditoría funcional
Aceptación y entrega	Revisión del producto final; Revisión de la documentación usuaria; Auditoría física

11. Planificación de la Gestión del Cambio

(Clase 12)

Conceptos clave

- **Versión:** instancia de un sistema que difiere de otras instancias. Si las diferencias son pequeñas → variante.
- **Entrega (release):** versión distribuida a los clientes. Siempre hay más versiones que entregas.

Esquema de versiones X.Y.Z (Versionado Semántico):

- **X** (versión mayor): nuevas funcionalidades importantes (nuevo módulo, característica clave).
- **Y** (versión menor): correcciones menores, errores arreglados, funcionalidades no cruciales.
- **Z** (revisión): cada entrega del proyecto por fallo corregido.

Ref: <https://semver.org/lang/es/>

Versiones por estabilidad:

- **Alpha:** versión inestable para prueba (ej. 1.0a1).
- **Beta:** más estable que Alpha, producto completo (ej. 2.0b).
- **RC (Release Candidate):** candidato a versión final (ej. 3.0-RC1).

Versiones por parche/fecha:

- Por parche: agrega un cuarto dígito (ej. 113.0.5672.93).
- Por fecha: Año.mes (2023.5), Año.mes.menor (2023.5.01).

Proceso de Gestión del Cambio:

1. Completar un **CRF (Change Request Form)** con: descripción del cambio, recomendaciones, costo estimado, fechas de solicitud/prueba/implementación/validación.
2. Registrar el CRF en la base de datos de configuraciones.
3. Analizar si el cambio es necesario, no duplicado y no ya considerado. Si es inválido → rechazar (notificar razón).
4. Evaluar y asignar costos; comprobar el impacto en el resto del sistema.
5. Considerar el impacto desde el punto de vista estratégico/organizacional y decidir si se justifica económicamente.
6. Implementar y registrar el cambio controladamente.

Base de datos de configuraciones (preguntas que debe responder):

1. ¿A qué clientes se les entregó una versión particular?
2. ¿Qué configuración de HW y SO requiere cada versión?
3. ¿Cuántas versiones existen y cuáles son sus fechas de creación?
4. ¿Qué versiones se ven afectadas si se cambia un componente?
5. ¿Cuántas peticiones de cambio están pendientes para una versión?
6. ¿Cuántos fallos declarados existen en una versión?

Entrega final — elementos incluidos:

- Documento de instalación y/o configuración.
 - Documentos del sistema (modelos de análisis, diseño, etc.).
 - Manual del usuario.
 - Código fuente.
-

12. Modelos Formales y Estándares

(Clase 13)

ISO 9000

Conjunto de estándares internacionales para sistemas de gestión de calidad en todas las industrias.

- **ISO 9001:** estándar más general; aplica a organizaciones interesadas en calidad de diseño, desarrollo y mantenimiento. No define procesos específicos de calidad (permite flexibilidad). *No garantiza la calidad del producto, solo que los procedimientos están definidos y documentados.*
 - **ISO 9000-3:** interpreta ISO 9001 para desarrollo de software.
 - **ISO 9004-2:** directrices para servicios de soporte de usuarios.
 - **ISO 27001:** seguridad de la información.
 - **ISO 20000:** servicios IT.
 - **ISO 25010:** atributos de calidad del software.
-

CMMI (Capability Maturity Model Integration)

Elaborado por el SEI (Software Engineering Institute) — Carnegie Mellon. Actualmente administrado por el Instituto CMMI.

Propósito: evaluar la madurez de los procesos de una organización y proporcionar orientación para mejorarlos, con el objetivo de mejorar los productos.

Modelo en Etapas (5 niveles de madurez):

Nivel	Nombre	Descripción
1	Inicial	Proceso impredecible, poco control
2	Administrado	Proceso por proyecto, documentación definida
3	Definido	Procesos estandarizados en toda la organización
4	Administrado cuantitativamente	Procesos medibles y predecibles
5	Optimizados	Mejora continua de los procesos

Modelo Continuo (6 niveles de capacidad):

Nivel	Descripción
0	Inmadura — sin implementación efectiva
1	Básica — se implementa y alcanzan los objetivos
2	Gestionada — se gestionan los procesos y los productos
3	Establecida — procesos definidos basados en estándares
4	Predecible — gestión cuantitativa de procesos
5	Optimizado — mejora continua para cumplir objetivos del negocio

Diferencia clave: el modelo en etapas define 5 niveles para la **organización completa**; el modelo continuo define 6 niveles por **área de proceso individual**.

SWEBOK (Software Engineering Body of Knowledge)

Guía del Cuerpo de Conocimientos de Ingeniería de Software. Describe el conocimiento generalmente aceptado sobre IS. Contiene **15 áreas de conocimiento (KA)**:

- 1. Requisitos de software
- 2. Diseño de software

3. Construcción de software
 4. Pruebas de software
 5. Mantenimiento de software
 6. Gestión de la configuración
 7. Gestión de la ingeniería de software
 8. Proceso de ingeniería de software
 9. Modelos y métodos de la IS
 10. Calidad del software
 11. Práctica profesional de la IS
 12. Economía de la IS
 13. Fundamentos de computación
 14. Fundamentos matemáticos
 15. Fundamentos de ingeniería
-

PMI (Project Management Institute)

Asociación mundial para la comunidad de profesionales de proyectos. Ofrece certificaciones relevantes:

- **PMP** (Project Management Professional)
 - **CAPM** (Certified Associate in Project Management)
 - **PMI-ACP** (Agile Certified Practitioner)
 - **PMI-RMP** (Risk Management Professional)
 - **DASM / DASSM** (Disciplined Agile Scrum Master)
-

RESUMEN DE FÓRMULAS CLAVE

Métrica	Fórmula
Especificidad Q1	$Q_1 = nu_i / nr$
Completitud Q2	$Q_2 = nc / (nc + nnv)$
Complejidad estructural	$S(i) = f_{out}(i)^2$
Complejidad de datos	$D(i) = v(i) / (f_{out}(i) + 1)$
Complejidad del sistema	$C(i) = S(i) + D(i)$

Métrica	Fórmula
Longitud Halstead	$N = N_1 + N_2$
Volumen Halstead	$V = N \times \log_2(n_1 + n_2)$
Esfuerzo Halstead	$E = V \times (n_1/2) \times (N_2/n_2)$
Tiempo Halstead	$T = E/18$ (segundos)
ERD	$ERD = E/(E + D)$
Tasa de éxito de pruebas	(pruebas exitosas / total pruebas) $\times$ 100
MTTR	$MTTR = T_d/N_r$
Integridad	$1 - (\text{amenaza} \times (1 - \text{seguridad}))$
IMS	$(M_t - F_c - F_a - F_d)/M_t$
UAW (UCP)	$\sum(\text{cant. actores} \times \text{peso})$
UUCW (UCP)	$\sum(\text{cant. CU} \times \text{peso})$
UUCP	$UAW + UUCW$
TCF	$0,6 + (0,01 \times TFactor)$
EF	$1,4 - (0,03 \times EFactor)$
AUCP	$UUCP \times TCF \times EF$
Esfuerzo total (UCP)	$E = AUCP \times CF$
MPC (CK)	$\sum C_i$ (suma de complejidades de métodos)

Resumen elaborado a partir de las clases 7 a 13 de Ingeniería de Software — Parcial 2.